Otimização do Sequenciamento na Produção de Baterias: Abordagem via RCPSP

Etapas 5 e 6: Resultados dos Métodos Exato e Aproximado

Cristian Madeira (Pós), Otávio Novais (162962), Thaís Melo (164934)

1 Objetivo

Este relatório reúne os resultados das Etapas 5 e 6. O sequenciamento das atividades na produção de baterias foi modelado como um RCPSP (*resource-constrained project scheduling problem*): minimizar o makespan $C_{\max}$ respeitando as precedências e a capacidade dos recursos renováveis. O problema foi resolvido de duas formas, nas mesmas instâncias: por um modelo exato de programação linear inteira mista (Gurobi) e pela metaheurística RKO (*Random-Key Optimizer*). A comparação procura responder a três perguntas:

1. Até que tamanho de instância o método exato entrega o ótimo com prova, e por que falha quando não entrega? (Seção 3)
2. Quanto se ganha, em tempo e em qualidade, ao trocar o método exato pelo aproximado? (Seção 4)
3. Dentro do método aproximado, o que mais importa: a estratégia de busca (GA, BRKGA ou RKO) ou o decodificador? (Seção 5)

2 Instâncias e configuração

Instâncias. Foram usadas 15 instâncias da PSPLIB [5], todas com quatro recursos e ótimo C^* conhecido:

- dez do J30 (30 atividades): j301_1, a instância apresentada na Etapa 3, e nove dos grupos mais difíceis, uma por grupo, todas com recursos escassos ($RS = 0{,}2$ ou $0{,}5$);
- cinco do J60 (60 atividades), com $RS = 0{,}2$, acrescentadas na Etapa 6 para avaliar o efeito do tamanho.

Os ótimos de referência foram obtidos com o CP-SAT (OR-Tools), com otimalidade provada. Nos grupos com $RS \geq 0{,}7$, o ótimo praticamente coincide com o limitante trivial (caminho crítico ou carga de recurso), e os métodos não se diferenciam; por isso, essas instâncias não foram usadas. Todos os experimentos rodaram no mesmo notebook (Intel Core i5-1135G7, 16 GB de RAM, Linux).

Método exato. Modelo tempo-indexado de Pritsker et al. [6], em AMPL, resolvido pelo Gurobi 13.0.3 com limite de 600 s por instância. Em relação à formulação da Etapa 4, foram adotados três refinamentos que não alteram o conjunto de soluções ótimas:

- **precedência desagregada**: $\sum_{s \leq t} x_{js} \leq \sum_{s \leq t - p_i} x_{is}$ para cada arco (i, j) e período t [2];
- **horizonte e janelas**: H é o melhor makespan do SSGS com sete regras de prioridade seguido de *forward-backward improvement* (FBI) [7]; a variável x_{jt} só existe para $t \in [ES_j, LS_j]$;
- **solução inicial**: o cronograma heurístico que define H é dado ao solver.

Cada solução devolvida foi conferida por uma rotina independente do modelo (precedência, capacidade e makespan), e todas foram aprovadas.

Método aproximado. RKO [1], na implementação C++ dos autores, sem alterações nas metaheurísticas. O decodificador é próprio do RCPSP: o vetor de chaves gera uma lista de prioridade topologicamente válida (a cada passo, a atividade elegível de menor chave), que o SSGS converte em cronograma e o FBI melhora. O RKO executa nove metaheurísticas em paralelo (BRKGA, SA, GRASP, ILS, VNS, PSO, GA, LNS e BRKGA-CS), que compartilham um conjunto de soluções de elite. Foram usados dois critérios de parada, ambos com 10 execuções por instância e encerramento ao atingir C^*:

- **tempo**: 10 s por execução no J30 e 30 s no J60;
- **orçamento**: 50.000 cronogramas (decodificações, somadas entre as threads), o critério usual da literatura [4].

Para separar o efeito da estratégia de busca, o GA e o BRKGA [3] do mesmo framework também foram executados isoladamente, com o mesmo decodificador, com e sem o FBI.

Métrica. A qualidade é medida pelo desvio relativo ao ótimo, RPD $= 100\,(C_{\max} - C^*)/C^*$. RPD $= 0$ significa que o ótimo foi encontrado.

3 Método exato

Tabela 1: Modelo tempo-indexado no Gurobi (600 s por instância).

Instância	RF	RS	Binárias	H	LP	$C_{\max}$	LB	C^*	Status	Tempo (s)
j301_1	0,25	0,2	482	46	38,8	**43**	43	43	ótimo	0,1
j309_2	0,75	0,2	1.879	98	63,0	**92**	92	92	ótimo	286,5
j3013_1	1,00	0,2	1.061	64	48,3	59	53	58	limite	600,1
j3025_4	0,75	0,2	1.515	89	55,3	**81**	81	81	ótimo	242,8
j3029_8	1,00	0,2	1.454	89	66,5	81	72	80	limite	600,1
j3037_7	0,50	0,2	1.603	98	58,3	**92**	92	92	ótimo	40,9
j3041_10	0,75	0,2	1.740	106	69,4	**99**	99	99	ótimo	165,7
j3045_6	1,00	0,2	2.290	133	89,1	**129**	124	129	limite	600,1
j3014_2	1,00	0,5	599	56	46,8	**53**	53	53	ótimo	51,0
j3030_2	1,00	0,5	663	70	65,0	**68**	68	68	ótimo	6,0
j605_4	0,50	0,2	2.700	83	58,3	77	66	72	limite	600,1
j609_4	0,75	0,2	2.821	94	72,9	92	83	87	limite	600,1
j6021_4	0,50	0,2	3.344	109	73,0	96	90	95	limite	600,1
j6037_2	0,50	0,2	2.673	100	72,3	96	88	95	limite	600,1
j6041_7	0,75	0,2	4.872	152	109,2	147	124	132	limite	600,2

RF, RS: fator e força de recurso do gerador ProGen. H: horizonte (heurística). LP: relaxação linear. $C_{\max}$: melhor solução inteira (em negrito quando igual a C^*). LB: melhor limitante inferior. Status: ótimo provado ou limite de tempo atingido.

J30: ótimo com prova na maioria das instâncias. O Gurobi provou a otimalidade em 7 das 10 instâncias, entre 0,1 s e 287 s (Tabela 1). Nas outras três, as soluções continuam boas: em j3045_6, o solver encontrou o ótimo (129), mas não o provou; em j3013_1 e j3029_8, terminou uma unidade acima do ótimo. O RPD médio no J30 é de 0,30%.

J60: nenhuma prova em 600 s. Nas cinco instâncias J60, o limite de tempo foi atingido, com gap entre 6,2% e 15,7% e RPD médio de 5,2%. O modelo tem de 2.700 a 4.900 binárias, contra 480 a 2.300 no J30, porque o número de variáveis acompanha o produto entre o número de atividades e o horizonte.

O tipo de dificuldade muda com o tamanho. A Figura 1 mostra de que lado do ótimo fica o intervalo em aberto. Nas três J30 não resolvidas, a solução está no ótimo ou a no máximo 1,7% dele, e quase todo o gap vem do limitante inferior (de 3,9% a 10% abaixo de C^*): o solver já tem a resposta, mas não consegue prová-la. No J60, o lado das soluções também falha: em j605_4 e j6041_7, a melhor solução fica 6,9% e 11,4% acima do ótimo, um erro maior que o do limitante (8,3% e 6,1% abaixo). Ou seja, nas instâncias maiores, o método exato tem dificuldade até para encontrar boas soluções.

Relaxação linear fraca. A relaxação linear fica, em média, 22% abaixo do ótimo no J30 e 20% no J60, e o solver precisa enumerar muitos nós para fechar essa distância. A fraqueza da relaxação, porém, não explica sozinha a dificuldade: j3037_7, com a pior relaxação (37% abaixo), foi resolvida em 41 s, enquanto j3013_1 (17% abaixo) não foi. As três J30 não resolvidas são exatamente as de $RF = 1{,}0$ e $RS = 0{,}2$, em que todas as atividades usam os quatro recursos e esses recursos são escassos, o que multiplica os conflitos de capacidade.

Papel da heurística. O horizonte H ficou, em média, 7% acima do ótimo no J30 e 12% no J60. O Gurobi melhorou essa solução inicial em todas as 15 instâncias, de modo que o método exato agrega valor sobre a heurística construtiva, mesmo quando não prova a otimalidade.

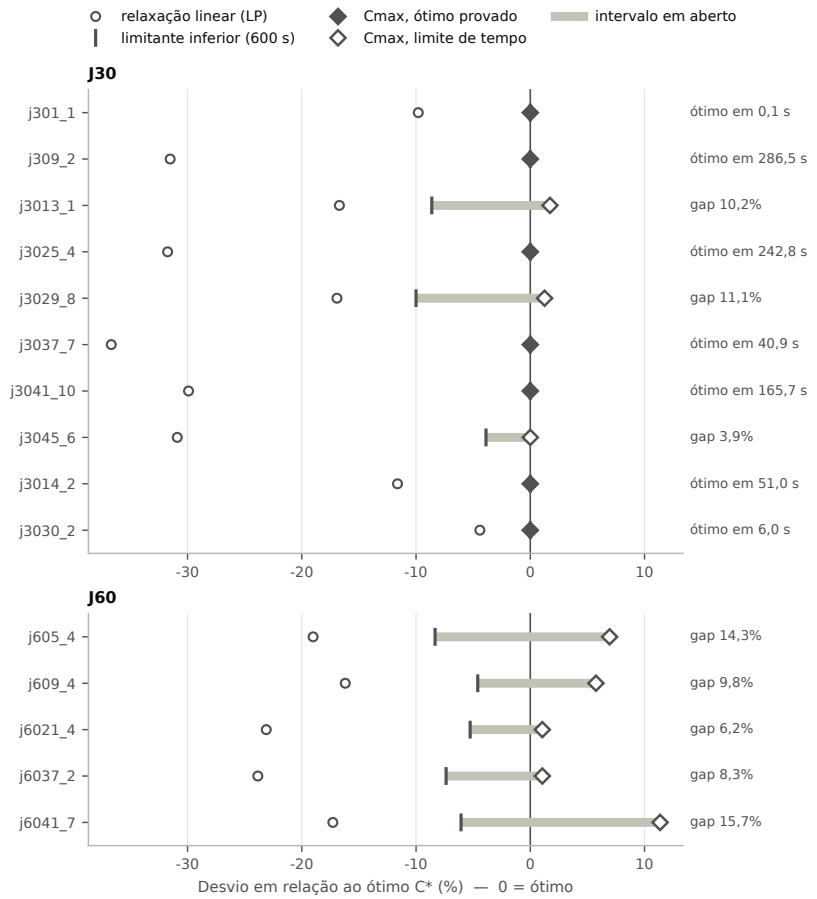


Figura 1: Posição da relaxação linear, do limitante inferior e da melhor solução do Gurobi em relação ao ótimo ($C^* = 0$). A barra cinza é o intervalo que o solver não fechou em 600 s; à direita, o gap final ou o tempo até a prova de otimalidade.

4 Exato × aproximado

Por instância. A Figura 2 compara os métodos instância a instância.

- **J30.** Seis das dez instâncias são resolvidas por todos os métodos, em todas as execuções. Nas demais, o RKO com tempo atinge o ótimo em j3013_1 (o Gurobi parou em 59), empata com o Gurobi em j3029_8 (81, uma unidade acima do ótimo, que nenhum método alcançou), empata em j3030_2 (ambos no ótimo) e fica atrás em j3025_4 (média de 81,8 contra o ótimo provado do Gurobi). Com 50.000 cronogramas, o RKO fica atrás do Gurobi nas quatro instâncias não triviais.
- **J60.** O RKO com tempo é melhor que o Gurobi em quatro instâncias e empata em j6037_2. Em j6041_7, a diferença é de 1,5% contra 11,4%. Mesmo com 50.000 cronogramas, o RKO supera o Gurobi em três das cinco instâncias.

Os traços mostram que a variação entre execuções do RKO é pequena com o critério de tempo (no máximo 1,7 ponto percentual entre a melhor e a pior execução) e maior com o orçamento fixo (até 3 pontos no J60).

Qualidade e tempo. A Figura 3 resume o compromisso entre os dois critérios:

- **No J30, a qualidade é a mesma, mas o RKO é cerca de 100 vezes mais rápido**: 0,28% em 2,6 s, contra 0,30% em 259 s do Gurobi. A vantagem que resta ao método exato é a prova de otimalidade, obtida em 7 das 10 instâncias.
- **No J60, o RKO é melhor nos dois critérios**: 1,0% em 24 s, contra 5,2% em 600 s.
- **O orçamento de 50.000 cronogramas** é o ponto mais rápido (menos de 1 s), ao custo de qualidade: 0,78% no J30 e 3,8% no J60, ainda melhor que o Gurobi neste último.

Em nenhum dos dois conjuntos o Gurobi é a melhor opção quando o objetivo é apenas uma boa solução:

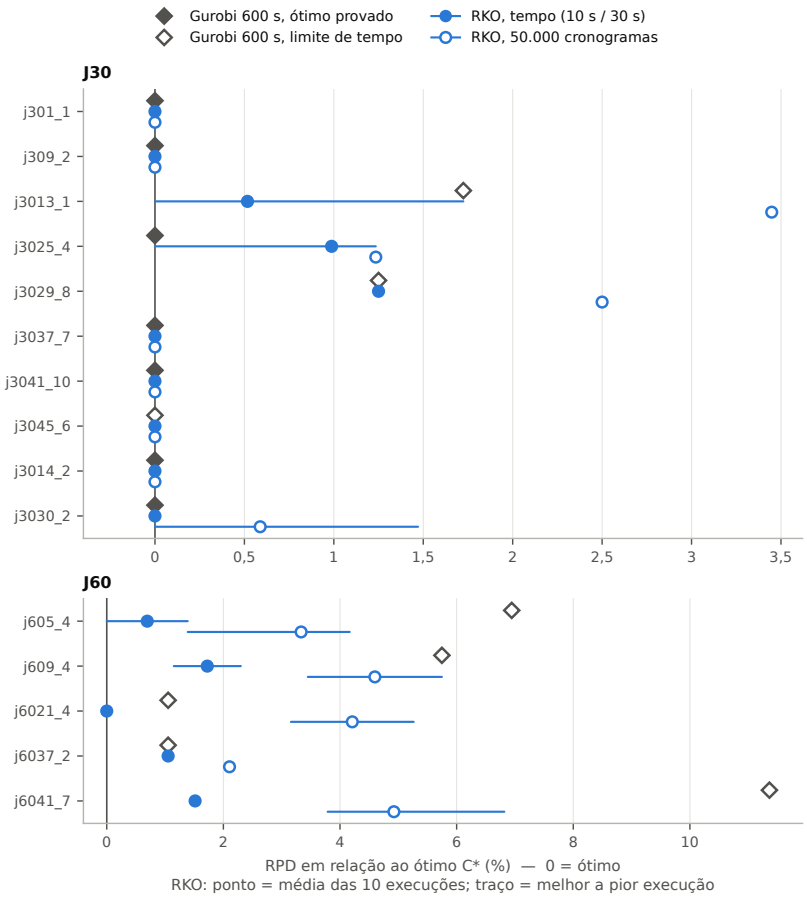


Figura 2: RPD por instância do Gurobi (600 s) e do RKO nos dois critérios de parada. RKO: média das 10 execuções (ponto) e intervalo entre a melhor e a pior execução (traço). Os eixos dos dois painéis têm escalas diferentes.

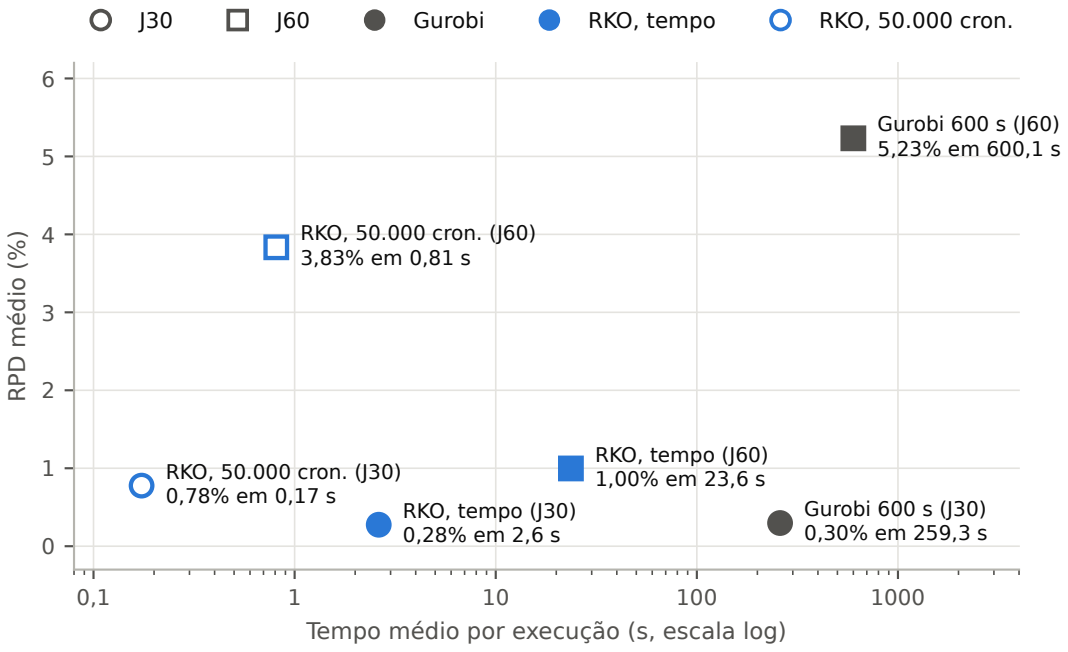


Figura 3: RPD médio × tempo médio por execução (escala logarítmica), por conjunto de instâncias. Pontos mais próximos do canto inferior esquerdo são melhores nos dois critérios.

o RKO com tempo é igual ou melhor em qualidade e de 25 a 100 vezes mais rápido. A vantagem do método aproximado cresce com o tamanho da instância, de acordo com o que se viu na Seção 3.

5 Estratégia de busca e decodificador

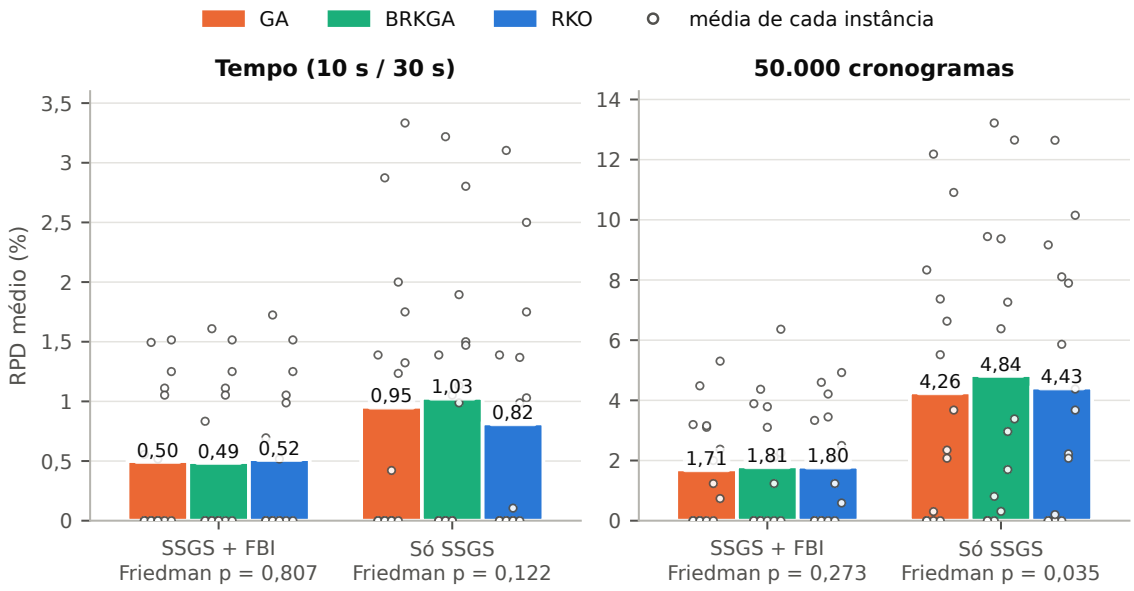


Figura 4: RPD médio nas 15 instâncias (10 execuções cada) do GA, do BRKGA e do RKO, com e sem o FBI no decodificador, nos dois critérios de parada. Os pontos são o RPD médio de cada instância; abaixo de cada grupo, o p do teste de Friedman entre os três métodos.

Para avaliar a observação de que algoritmos genéticos têm dificuldade neste problema, o GA e o BRKGA foram comparados ao RKO com a mesma representação e o mesmo decodificador (Figura 4).

A estratégia de busca pouco importa. Com o FBI, os três métodos são estatisticamente equivalentes nos dois critérios (Friedman: $p = 0{,}81$ com tempo e $p = 0{,}27$ com orçamento). A única diferença significativa aparece sem o FBI e com orçamento fixo ($p = 0{,}035$): o GA supera o BRKGA (Wilcoxon pareado com correção de Holm, $p = 0{,}02$), e o RKO fica entre os dois. Os pontos da figura reforçam a conclusão: a diferença entre instâncias (de 0 a 13% de RPD) é muito maior que a diferença entre métodos (menos de 0,6 ponto percentual). A dificuldade esperada do GA, portanto, não se confirmou. Uma explicação provável para o desempenho inferior do BRKGA sem FBI é o orçamento: o framework divide os cronogramas entre nove cópias paralelas, e, com a população padrão de 987 indivíduos, cada cópia do BRKGA faz só 3 a 4 gerações, contra 8 a 11 do GA (população de 400).

O decodificador é o fator de maior efeito. O FBI reduz o RPD médio de 4,3–4,8% para 1,7–1,8% com orçamento fixo e de 0,8–1,0% para cerca de 0,5% com tempo. Há uma ressalva na comparação por orçamento: com o FBI, cada cronograma contado envolve, em média, 5,3 a 5,6 passadas do SSGS (a original e as de justificação), e por isso o FBI avalia cerca de 5 vezes menos cronogramas por segundo (90 mil contra 500 mil). A comparação por tempo, que desconta esse custo, mantém a conclusão: mesmo avaliando menos cronogramas, o FBI reduz o RPD no J60 de 1,7–2,1% para 0,9–1,0% (Mann-Whitney, $p < 0{,}01$ nos três métodos). No J30, a diferença é pequena (0,24–0,29% contra 0,38–0,51%), porque a maioria das execuções já chega ao ótimo.

6 Limitações

- Foram usadas 15 instâncias, escolhidas entre as de recursos escassos. Os resultados não se estendem diretamente às instâncias fáceis ($RS \geq 0{,}7$) nem ao J90 e ao J120.
- Os parâmetros das metaheurísticas são os padrão do framework e não foram calibrados para o orçamento de 50.000 cronogramas, o que limita a comparação entre GA e BRKGA.

- Os tempos foram medidos em uma única máquina, e os dois métodos usam paralelismo de formas diferentes (o RKO, nove metaheurísticas em threads separadas; o Gurobi, o *branch-and-bound* paralelo), o que torna a razão entre os tempos apenas indicativa.

7 Conclusões

- **Instâncias pequenas (J30):** o modelo exato entrega o ótimo com prova em 7 de 10 instâncias, a um custo de minutos. Quando falha, a solução é boa e o que falta é a prova (limitante inferior fraco nas instâncias com $RF = 1{,}0$ e $RS = 0{,}2$).
- **Instâncias maiores (J60):** o modelo exato não prova nenhum ótimo em 600 s e passa a errar também nas soluções (RPD médio de 5,2%). O RKO obtém 1,0% em 24 s.
- **Metaheurística:** com o mesmo decodificador, GA, BRKGA e RKO são equivalentes; o que decide a qualidade é o FBI no decodificador.

Os dois métodos são complementares. O RKO é a escolha para gerar boas soluções rapidamente, e o método exato, para certificar o ótimo em instâncias pequenas. Uma combinação natural para o artigo é usar a solução do RKO como horizonte e solução inicial do modelo exato: um H mais justo estreita as janelas $[ES_j, LS_j]$ e reduz o número de binárias, o que pode tornar o J60 tratável pelo Gurobi. O RKO será a metaheurística do artigo, com o GA e o BRKGA como referência.

Referências

[1] Chaves, A. A. et al. A random-key optimizer for combinatorial optimization. *Journal of Heuristics*, 31(4), 2025.

[2] Christofides, N.; Alvarez-Valdés, R.; Tamarit, J. M. Project scheduling with resource constraints: a branch and bound approach. *European Journal of Operational Research*, 29(3):262–273, 1987.

[3] Gonçalves, J. F.; Resende, M. G. C. Biased random-key genetic algorithms for combinatorial optimization. *Journal of Heuristics*, 17(5):487–525, 2011.

[4] Kolisch, R.; Hartmann, S. Experimental investigation of heuristics for resource-constrained project scheduling: an update. *European Journal of Operational Research*, 174(1):23–37, 2006.

[5] Kolisch, R.; Sprecher, A. PSPLIB – a project scheduling problem library. *European Journal of Operational Research*, 96(1):205–216, 1997.

[6] Pritsker, A. A. B.; Watters, L. J.; Wolfe, P. M. Multiproject scheduling with limited resources: a zero-one programming approach. *Management Science*, 16(1):93–108, 1969.

[7] Valls, V.; Ballestín, F.; Quintanilla, S. Justification and RCPSP: a technique that pays. *European Journal of Operational Research*, 165(2):375–386, 2005.